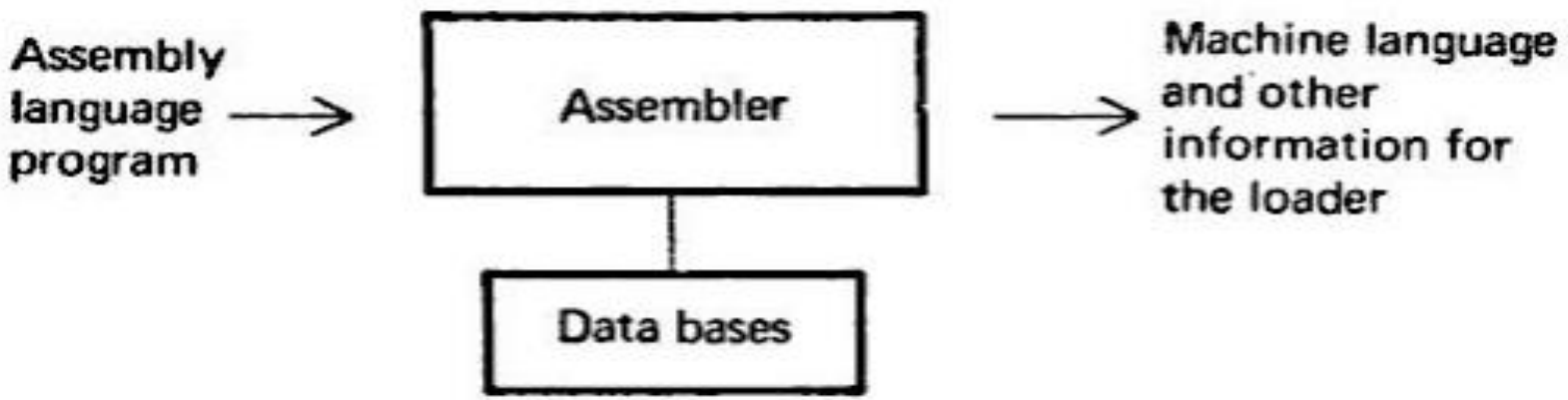
Unit 2

Assembler

Assembly Language

- An assembler is a program that takes computer instruction and converts them into a pattern of bits that the computer processor can use to perform its basic operation.
- The assembler is responsible for translating the assembly language program into machine code. When the source program language is essentially a symbolic representation for a numerical machine language, the translator is called assembler and the source language is called an assembly language.



Basic function of Assembler

- Translate mnemonics opcodes to machine language.
- Convert symbolic operands to their machine addresses.
- Build machine instructions in the proper format
- Convert data constants into machine representation.
- Error checking is provided.
- Changes can be quickly and easily incorporated with a reassembly.
- Variables are represented by symbolic names, not as memory locations.
- Assembly language statements are written one per line. A machine code program thus consists of a sequence of assembly language statements, where each statement contains a mnemonics.

Advantages

- Reduced errors
- Faster translation times
- Changes could be made easier and faster.
- Addresses are symbolic, not absolute \
- Easy to remember

Disadvantages

- Assembler language are unique to specific types of computer
- Program is not portable to the computer.
- Many instructions are required to achieve small tasks
- Programmer required knowledge of the processor architecture and instruction set.

General Design Procedure

Translation phase of Assembler

The six steps that should be followed by the designer

1. Specify the problem
2. Specify data structure
3. Define format of data structure
4. Specify algorithm
5. Look for modularity
6. Repeat 1 through 5 on modules

3.2 DESIGN OF ASSEMBLER

3.2.1 Statement of Problem

Let us pretend that we are the assembler trying to translate the program in the left column of Figure 3.2. We read the START instruction and note that it is a pseudo-op instruction (to the assembler) giving JOHN as the name of this program; the assembler must pass the name on to the loader. The next instruction is the USING pseudo-op. This tells the assembler (us) that register 15 is the base register and at execution time will contain the address of the first instruction of the program.

Source program			First pass			Second pass		
			Relative address	Mnemonic instruction		Relative address	Mnemonic instruction	
JOHN	START	0						
	USING	*,15						
	L	1,FIVE	0	L	1,-(0,15)	0	L	1,16(0,15)
	A	1,FOUR	4	A	1,-(0,15)	4	A	1,12(0,15)
	ST	1,TEMP	8	ST	1,-(0,15)	8	ST	1,20(0,15)
FOUR	DC	F'4'	12	4		12	4	
FIVE	DC	F'5'	16	5		16	5	
TEMP	DS	1F	20	-		20	-	
	END							

FIGURE 3.2 Intermediate steps in assembling a program

There is no BALR instruction. This program was presumably called by another program that left the address of the first instruction in register 15 (see standard subroutine linkage conventions in Appendix B). Next comes a Load instruction: L 1, FIVE. We can look up the bit configuration for the mnemonic in a table and put the bit configuration for the L in the appropriate place in the machine language instruction. Now we need the address of FIVE. At this point, however, we do not know where FIVE is, so we cannot supply its address. Because no index register is being used, we put in 0 for the index register. We know that register 15 is being used as the base register, but we cannot calculate the offset. The base register 15 is pointing to the beginning of this program, and the offset is going to be the difference between the location FIVE and the location of the beginning of the program, which is not known at this time. We maintain a *location counter* indicating the relative address of the instruction being processed; this counter is incremented by 4 (length of a Load instruction).

The next instruction is an Add instruction. We look up the op-code, but we do not know the offset for FOUR. The same thing happens with the Store instruction. The DC instruction is a pseudo-op directing us to define some data; for FOUR we are to produce a '4'. We know that this word will be stored at relative location 12, because the location counter now has the value 12, having been incremented by the length of each preceding instruction. The first instruction, four bytes long, is at relative address 0. The next two instructions are also four bytes long. We say that the symbol "FOUR" has the value 12. The next instruction has as a label FIVE and an associated location counter value of 16. The label on TEMP has an associated value of 20. We have thus produced the code in the center column of Figure 3.2

As the assembler, we can now go back through the program and fill in the offsets as is done in the third column of Figure 3.2.

Because symbols can appear before they are defined, it is convenient to make two passes over the input (as this example shows). The first pass has only to define the symbols; the second pass can then generate the instructions and addresses. (There are one-pass assemblers and multiple-pass assemblers. Their design and implications are discussed in this chapter). Specifically, an assembler must do the following:

1. Generate instructions:
   a. Evaluate the mnemonic in the operation field to produce its machine code.
   b. Evaluate the subfields — find the value of each symbol, process literals, and assign addresses.
2. Process pseudo ops

An assembler must do the following

1. Generate instruction
    a. Evaluate the mnemonics in the operation field to produce the machine code
    b. Evaluate the subfield-fine the value of each symbol. Process literals and assign addresses.
2. Process pseudo ops
    a. Pass 1 (Define symbol and literals)
        i. Determine length of machine instruction (MOTGET)
        ii. Keep track of location counter (LC)
        iii. Remember value of symbol until pass 2 (STSTO)
        iv. Process some pseudo ops(POTGET1)
        v. Remember literal (LITSTO)
    b. Pass 2 (Generate object Program)
        i. Look up value of symbol (STGET)
        ii. Generate instruction (MOTGET2)
        iii. Generate data (for DS, DC and Literal)
        iv. Process pseudo ops (POTGET2)

Design data structure for assembler design in Pass-1 and Pass-2 with flow chart

Pass -1

1. Input source program
2. A location counter used to keep track of each instruction location.
3. A table, the machine –operation table (MOT) that indicate the symbolic mnemonics for each instruction and its length (tow, four or six bytes)
4. A table, the pseudo operation table (POT) that indicate the symbolic mnemonics and action to be taken for each pseudo-op in pass-1
5. A table, the literal table (LT) that is used to store each literal encounter and its corresponding assigned location.
6. A table, the symbol table (ST) that is used to store each label and its corresponding value.
7. A copy of the input to be used later by Pass-2. This may be stored in a secondary storage device

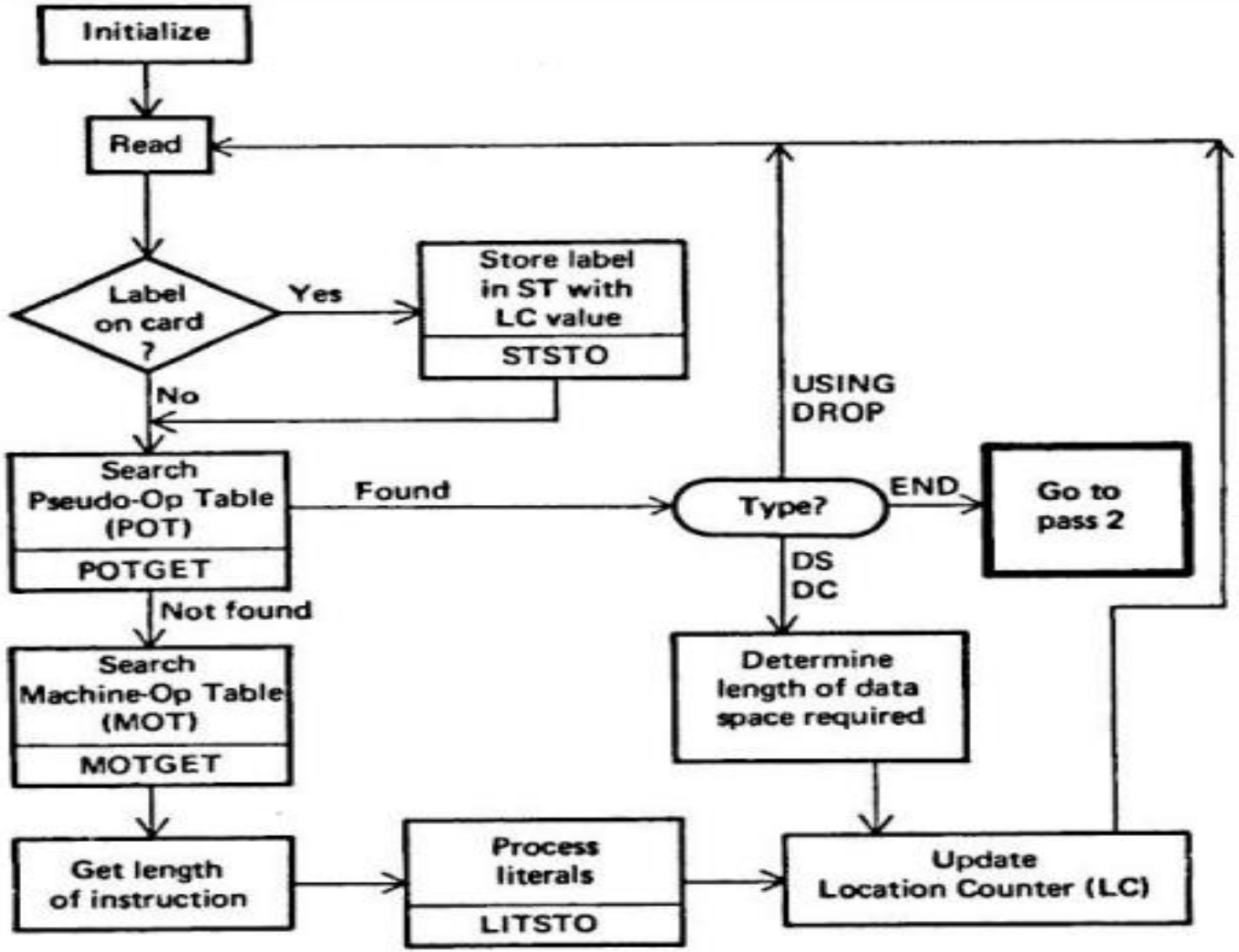


FIGURE 3.3 Pass 1 overview: define symbols

Pass-2

1. Copy of source program input to pass-1
2. Location counter
3. A table the MOT that indicates for each instruction
    a. Symbolic
    b. Mnemonics
    c. Length
    d. Binary machine op-code
    e. Format (RR, RS, RX, SI, SS)

4. A table the POT that indicates for each pseudo-op the symbolic mnemonic and the action to be taken in Pass-2
5. The ST prepare by Pass-1, containing each label and its corresponding value.
6. A table, BT that indicate which register are currently specified by base register by USING pseudo-ops and what are the specified contents of these register.
7. A work space INSR that is to hold each instruction as its various parts are being assembled together.
8. A workspace PRINT LINE used to produce a printed listing
9. A workspace PUNCH CARD used prior to actual outputting for converting assembled instruction into the format needed by the loader.
10. An output deck of assembled instruction in the format needed by the loader.

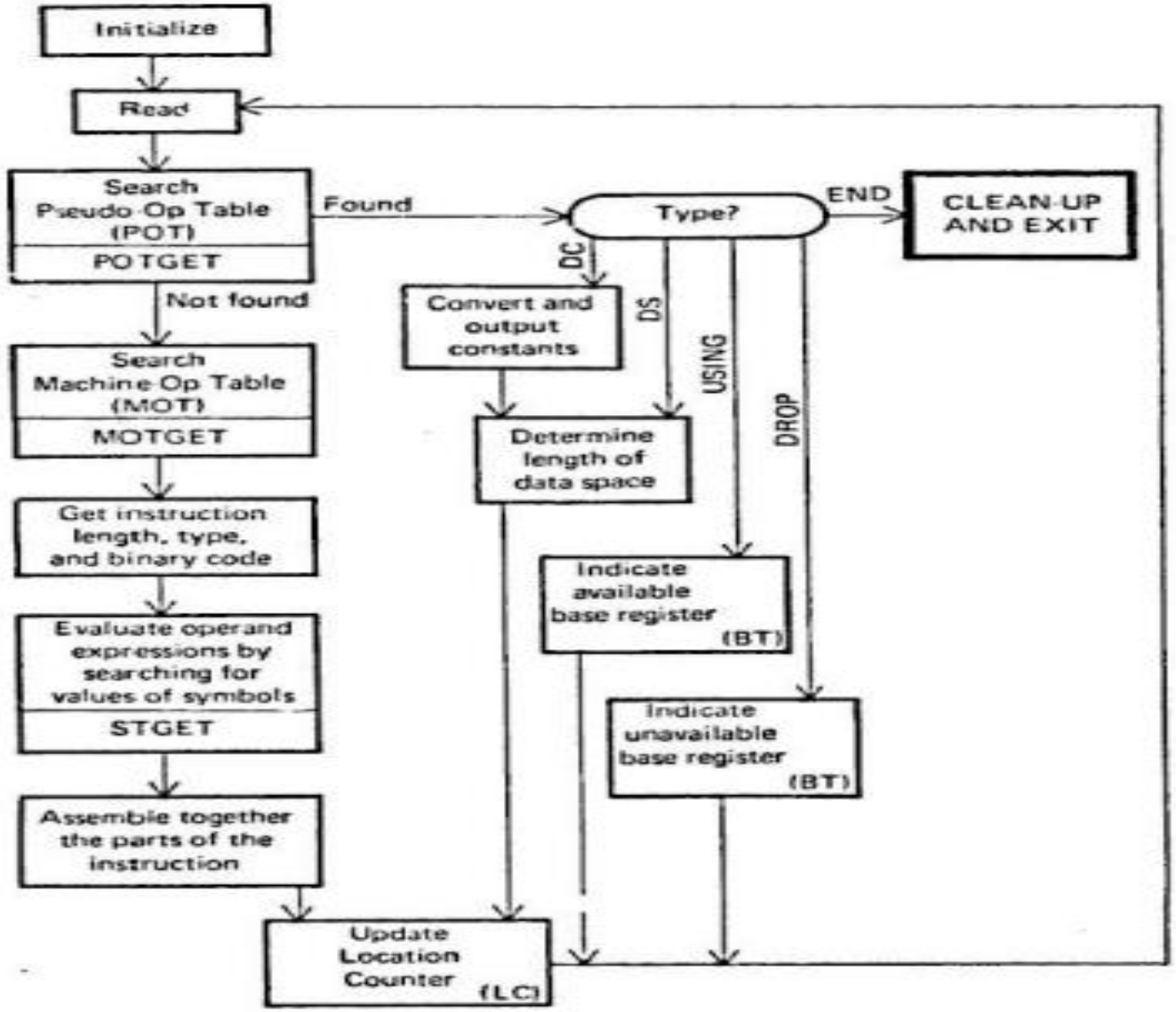


FIGURE 3.4 Pass 2 overview: evaluate fields and generate code

3.2.3 Format of data Bases

1. Machine Operation Table (MOT)

Fig machine op table for pass-1 and pass-2 the op code is the key and its value is the binary op-code equivalent which is stored for use in generating machine op-code. The instruction length is stored for use in updating the location counter, the instruction format for use in forming the machine language equivalent.

6-bytes per entry

Mnemonic op-code (4-bytes) (characters)	Binary op-code (1-byte) (hexadecimal)	Instruction length (2-bits) (binary)	Instruction format (3-bits) (binary)	Not used in this design (3-bits)
"A*bbb*"	5A	10	001	
"AH*bb*"	4A	10	001	
"AL*bb*"	5E	10	001	
"ALR*b*"	1E	01	000	
"AR*bb*"	1A	01	000	
...	...	...	...	
"MVC*b*"	D2	11	100	
...	...	...	...	

b ~ represents the character "blank"

Codes:

Instruction length

01 = 1 half-words = 2 bytes
10 = 2 half-words = 4 bytes
11 = 3 half-words = 6 bytes

Instruction format

000 = RR
001 = RX
010 = RS
011 = SI
100 = SS

2. Pseudo Operation Table (POT)

The table will actually contain the physical address. Fig POT for pass-1, each pseudo-op is listed with and associated pointer to the assembler routine for processing the pseudo-op.

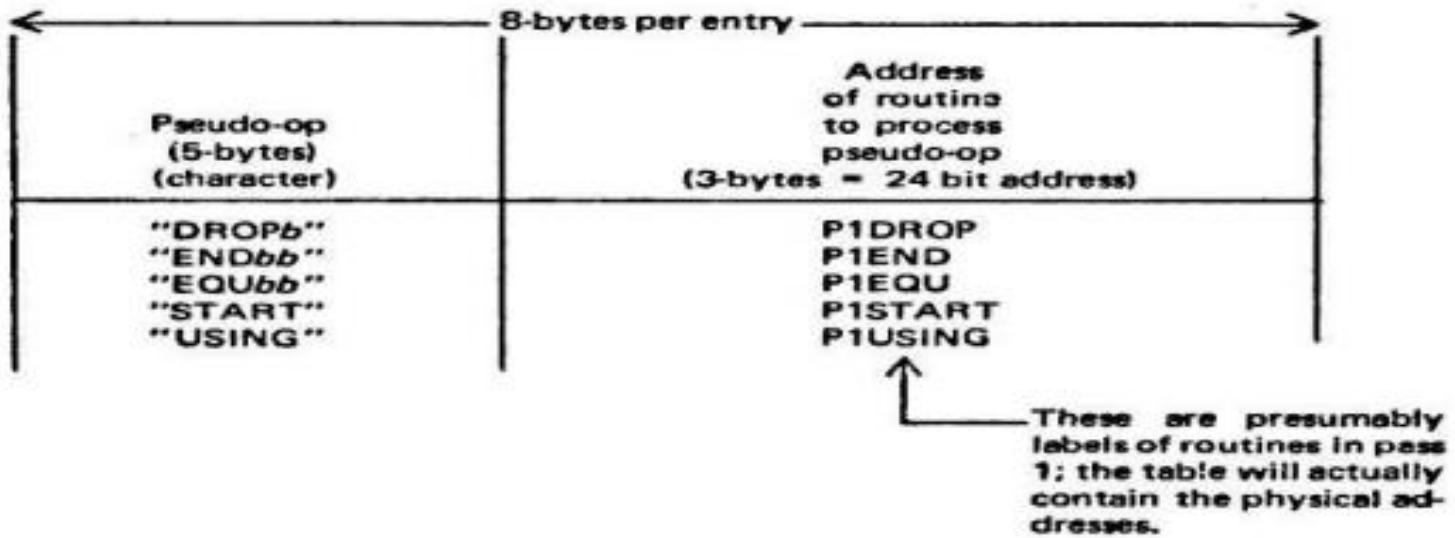


FIGURE 3.7 Pseudo-Op Table (POT) for pass 1 (similar table for pass 2)

3. Symbol Table (ST)

Symbol table is usually hash-organized. It contains all relevant information about symbol defined and used in the source program. Information regarding all forward references in a symbol table, for each symbol and address is generated randomly and the information regarding that symbol get the same address conflict is resolve by using collision handling techniques.

The relative location indicator tells the assembler whether the value of the symbol is absolute or relative base of the program.

14-bytes per entry

Symbol (8-bytes) (characters)	Value (4-bytes) (hexadecimal)	Length (1-byte) (hexadecimal)	Relocation (1-byte) (character)
"JOHN*bbbb*"	0000	01	"R"
"FOUR*bbbb*"	000C	04	"R"
"FIVE*bbbb*"	0010	04	"R"
"TEMP*bbbb*"	0014	04	"R"

FIGURE 3.8 Symbol Table (ST) for pass 1 and pass 2

4. Base Table (BT)

Base table is used by the assembler to generate the proper base register references in machine instruction and to compute the correct offsets, when generating an address, the assembler referencing the base register table to chose a base register that will contains a value close to the symbolic references. The address is the information using that base register.

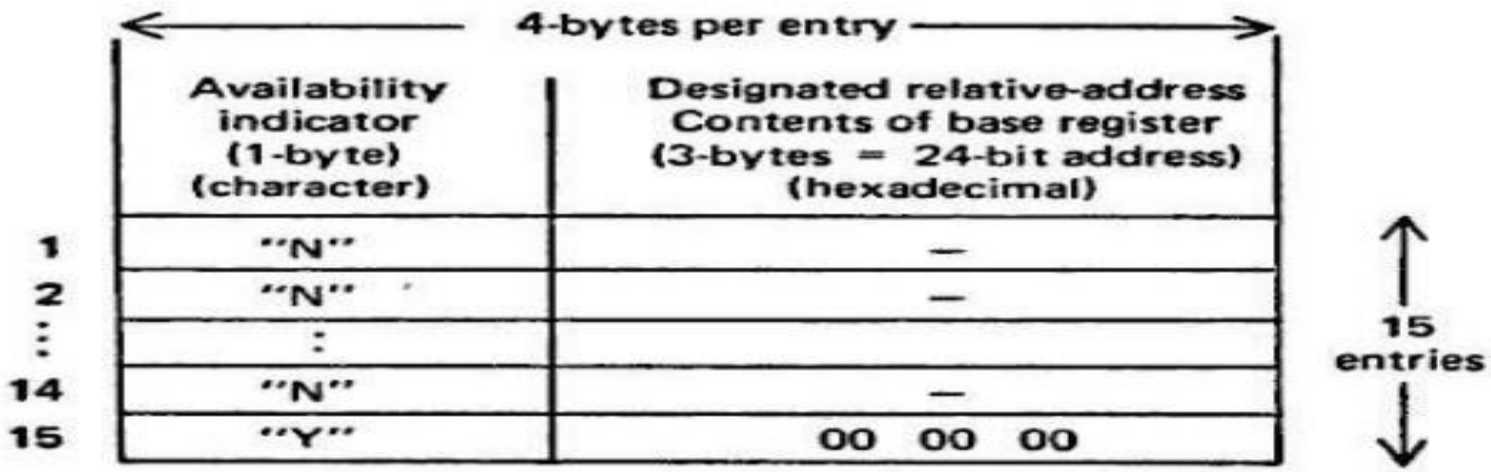


FIGURE 3.9 Base Table (BT) for pass 2

3.2.4 Algorithm

The flowcharts in Figures 3.10 and 3.11 describe in some detail an algorithm for an assembler for an IBM 360 computer. These diagrams represent a simplification of the operations performed in a complex assembler but they illustrate most of the logical processes involved.

PASS 1: DEFINE SYMBOLS

The purpose of the first pass is to assign a location to each instruction and data-defining pseudo-instruction, and thus to define values for symbols appearing in the label fields of the source program. Initially, the Location Counter (LC) is set to the first location in the program (relative address 0). Then a source statement is read. The operation-code field is examined to determine if it is a pseudo-op; if it is not, the table of machine op-codes (MOT) is searched to find a match for the source statement's op-code field. The matched MOT entry specifies the length (2,4 or 6 bytes) of the instruction. The operand field is scanned for the presence of a literal. If a new literal is found, it is entered into the Literal Table (LT) for later processing. The label field of the source statement is then examined for the presence of a symbol. If there is a label, the symbol is saved in the Symbol Table (ST) along with the current value of the location counter. Finally, the current value of the location counter is incremented by the length of the instruction and a copy of the source card is saved for use by pass 2. The above sequence is then repeated for the next instruction.

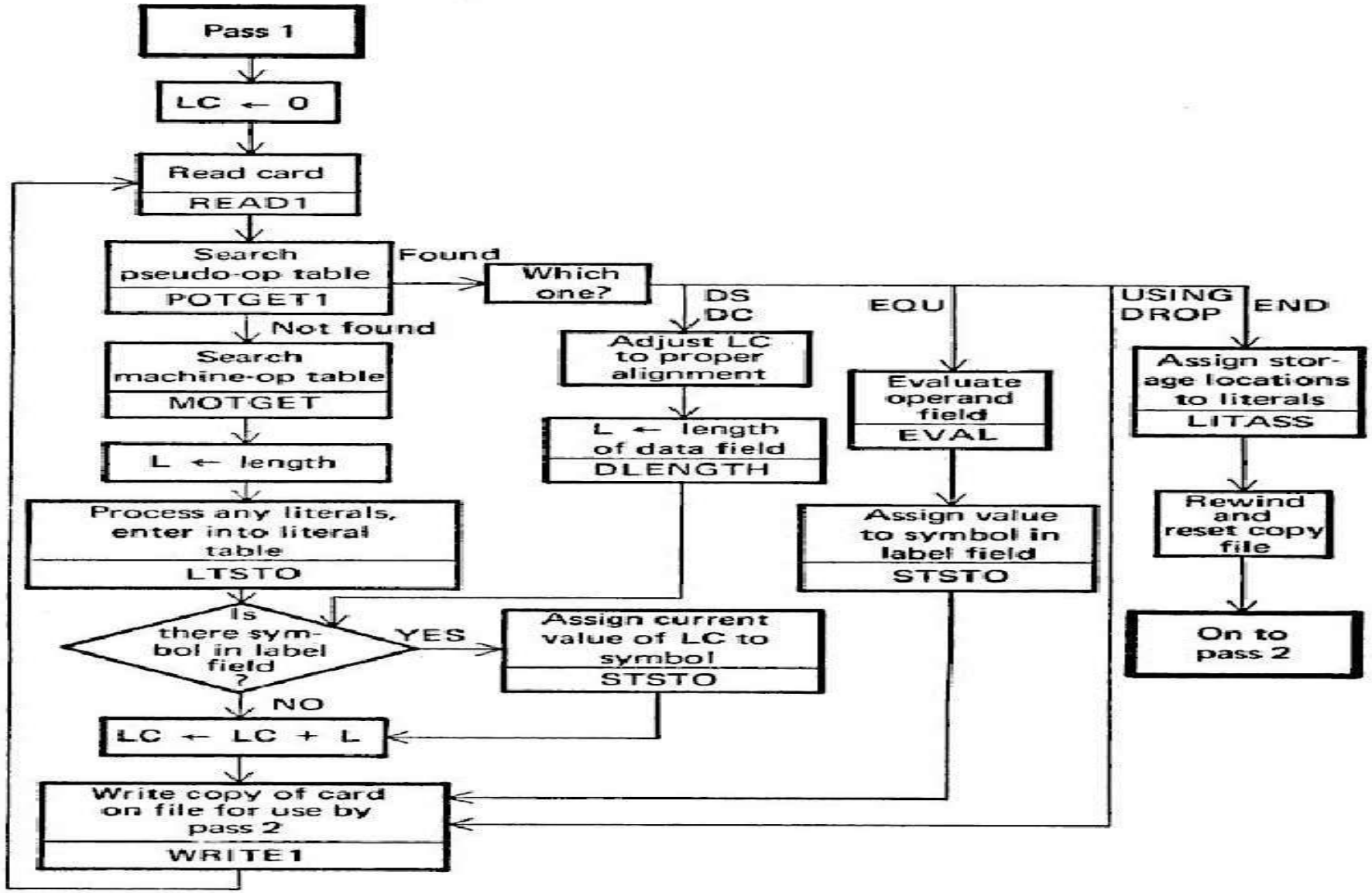


FIGURE 3.10 Detailed pass 1 flowchart

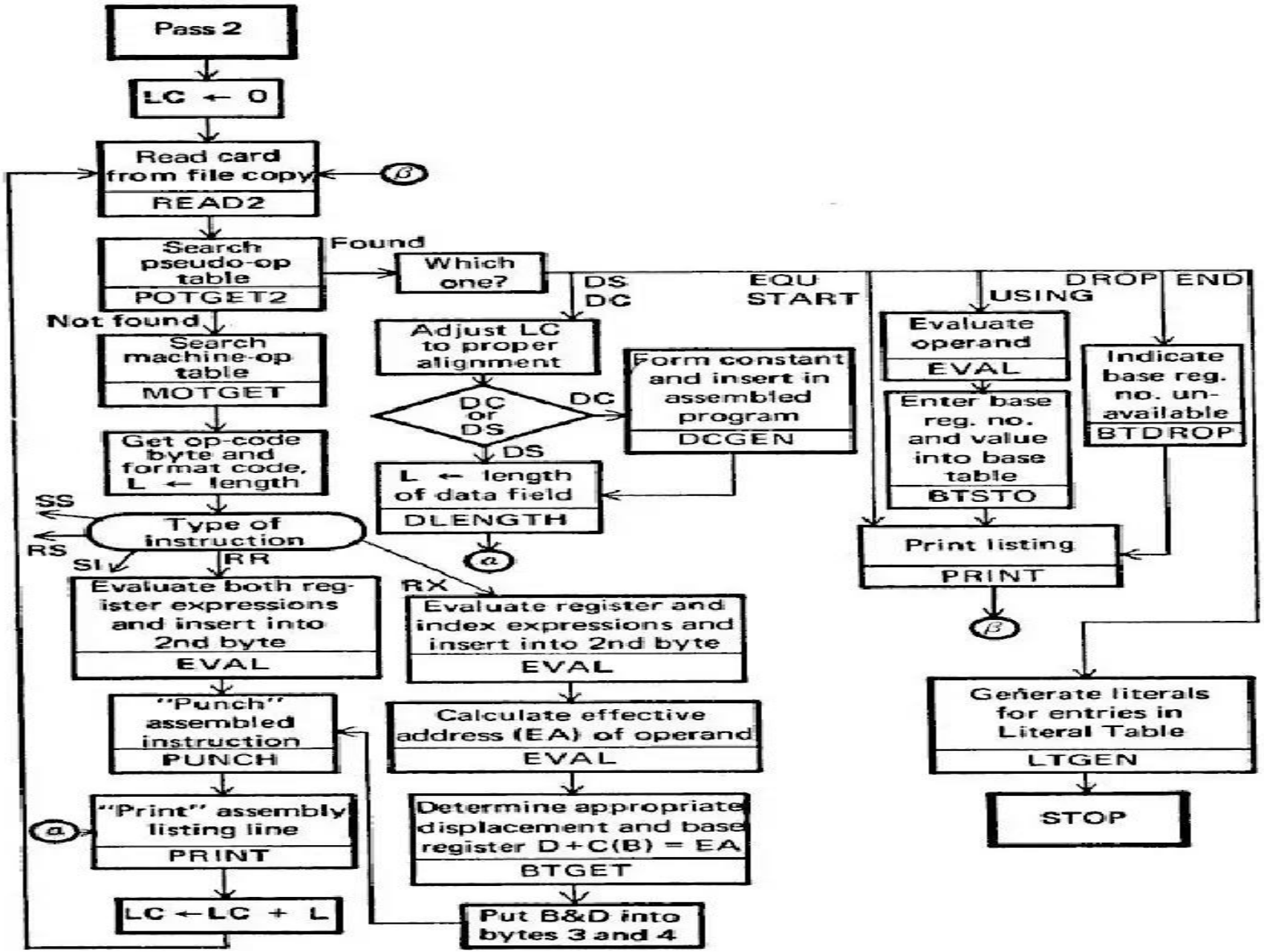


FIGURE 3.11 Detailed pass 2 flowchart

The loop described is physically a small portion of pass 1 even though it is the most important function. The largest sections of pass 1 and pass 2 are devoted to the special processing needed for the various pseudo-operations. For simplicity, only a few major pseudo-ops are explicitly indicated in the flowchart (Fig. 3.10); the others are processed in a straightforward manner.

We now consider what must be done to process a pseudo-op. The simplest procedure occurs for USING and DROP. Pass 1 is only concerned with pseudo-ops that define symbols (labels) or affect the location counter; USING and DROP do neither. The assembler need only save the USING and DROP cards for pass 2.

In the case of the EQU pseudo-op during pass 1, we are concerned only with defining the symbol in the label field. This requires evaluating the expression in the operand field. (The symbols in the operand field of an EQU statement must have been defined previously.)

The DS and DC pseudo-ops can affect both the location counter and the definition of symbols in pass 1. The operand field must be examined to determine the number of bytes of storage required. Due to requirements for certain alignment conditions (e.g., fullwords must start on a byte whose address is a multiple of four), it may be necessary to adjust the location counter before defining the symbol.

When the END pseudo-op is encountered, pass 1 is terminated. Before transferring control to pass 2, there are various "housekeeping" operations that must be performed. These include assigning locations to literals that have been collected during pass 1, a procedure that is very similar to that for the DC pseudo-op. Finally, conditions are reinitialized for processing by pass 2.

PASS 2: GENERATE CODE

After all the symbols have been defined by pass 1, it is possible to finish the assembly by processing each card and determining values for its operation code and its operand field. In addition, pass 2 must structure the generated code into the appropriate format for later processing by the loader, and print an assembly listing containing the original source and the hexadecimal equivalent of the bytes generated. The Location Counter is initialized as in pass 1, and the processing continues as follows.

A card is read from the source file left by pass 1. As in pass 1, the operation code field is examined to determine if it is a pseudo-op; if it is not, the table of machine op-codes (MOT) is searched to find a match for the card's op-code field. The matching MOT entry specifies the length, binary op-code, and the format-type of the instruction. The operand fields of the different instruction format types require somewhat different processing.

For the RR-format instructions, each of the two register specification fields is evaluated. This evaluation may be very simple, as in:

```
AR      2,3
```

or more complex, as in:

```
MR      EVEN,EVEN+1
```

The two fields are inserted into their respective four-bit fields in the second byte of the RR-instruction.

For RX-format instructions, the register and index fields are evaluated and processed in the same way as the register specifications for RR-format instructions. The storage address operand is evaluated to generate an Effective Address (EA). Then the base register table (BT) must be examined to find a suitable base register (B) such that $D = EA - c(B) < 4096$. The corresponding displacement can then be determined. The 4-bit base register specification and 12-bit displacement fields are then assembled into the third and fourth bytes of the instruction. Only the RR and RX instruction types are explicitly shown in the flowchart (Fig. 3.11). The other instruction formats are handled similarly.

After the instruction has been assembled, it is put into the necessary format for later processing by the loader. Typically, several instructions are placed on a single card (see Chapter 5 for a more detailed discussion). A listing line containing a copy of the source card, its assigned storage location, and its hexadecimal representation is then printed. Finally, the location counter is incremented and processing is continued with the next card.

As in pass 1, each of the pseudo-ops calls for special processing. The EQU pseudo-op requires very little processing in pass 2, because symbol definition was completed in pass 1. It is necessary only to print the EQU card as part of the printed listing.

The USING and DROP pseudo-ops, which were largely ignored in pass 1, require additional processing in pass 2. The operand fields of the pseudo-ops are evaluated; then the corresponding Base Table entry is either marked as available, if USING, or unavailable, if DROP. The base table is used extensively in pass 2 to compute the base and displacement fields for machine instructions with storage operands.

The DS and DC pseudo-ops are processed essentially as in pass 1. In pass 2, however, actual code must be generated for the DC pseudo-op. Depending upon the data types specified, this involves various conversions (e.g., floating point character to binary representation) and symbol evaluations (e.g., address constants).

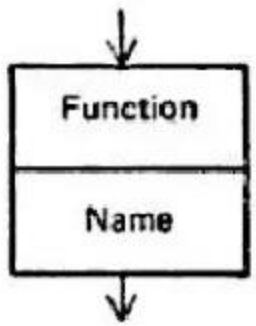
The END pseudo-op indicates the end of the source program and terminates the assembly. Various "housekeeping" tasks must now be performed. For example, code must be generated for any literals remaining in the Literal Table (LT).

3.2.5 Look for Modularity

We now review our design, looking for functions that can be isolated. Typically,

such functions fall into two categories: (1) multi-use and (2) unique.

In the flowcharts for pass 1 (Fig. 3.10) and pass 2 (Fig. 3.11), we examine each step as a candidate for logical separation. Likely choices are identified in the flowcharts by the shape



where "name" is the name assigned to the function (e.g., MOTGET, EVAL, PRINT).

Listed below are some of the functions that may be isolated in the two passes.

PASS 1:

1. READ1	——	Read the next assembly source card.
2. POTGET1	——	*Search* the pass 1 Pseudo-Op Table (POT) for a match with the operation field of the current source card.
3. MOTGET1	——	*Search* the Machine-Op Table (MOT) for a match with the operation of the current source card.
4. STSTO	——	*Store* a label and its associated value into the Symbol Table (ST). If the symbol is already in the table, return error indication (multiply-defined symbol).
5. LTSTO	——	*Store* a literal into the Literal Table (LT); do not store the same literal twice.
6. WRITE1	——	Write a copy of the assembly source card on a storage device for use by pass 2.

7. DLENGTH	---	*Scan* operand field of DS or DC pseudo-op to determine the amount of storage required.
8. EVAL	---	*Evaluate* an arithmetic expression consisting of constants and symbols (e.g. 6, ALPHA, BETA + 4 * GAMMA).
9. STGET	---	*Search* the Symbol Table (ST) for the entry corresponding to a specific symbol (used by STSTO, and EVAL).
10. LITASS	---	Assign storage locations to each literal in the literal table (may use DLENGTH).

PASS 2:

1. READ2	———	Read the next assembly source card from the file copy.
2. POTGET2	———	Similar to POTGET1 (search POT).
3. MOTGET2	———	Same as in pass 1 (search MOT).
4. EVAL	———	Same as in pass 1 (evaluate expressions).
5. PUNCH	———	Convert generated instruction to card format; punch card when it is filled with data.
6. PRINT	———	Convert relative location and generated code to character format; print the line along with copy of the source card.
7. DCGEN	———	Process the fields of the DC pseudo-op to generate object code (uses EVAL and PUNCH).
8. DLENGTH	———	Same as in pass 1.

9. BTSTO	---	Insert data into appropriate entry of Base Table (BT).
10. BTDROP	---	Insert "unavailable" Indicator into appropriate entry of BT.
11. BTGET	---	Convert effective address into base and displacement by searching Base Table (BT) for available base registers.
12. LTGEN	---	Generate code for literals (uses DCGEN).

Each of these functions should independently go through the entire design process (problem statement, data bases, algorithm, modularity, etc.). These functions can be implemented as separate external subroutines, as internal subroutines, or as sections of the pass 1 and pass 2 programs. In any case, the ability to treat functions separately makes it much easier to design the structure of the assembler and each of its parts. Thus, rather than viewing the assembler as a single program (of from 1,000 to 10,000 source statements typically), we view it as a coordinated collection of routines each of relatively minor size and complexity.

We will not attempt to examine all of these functional routines in detail since they are quite straightforward. There are two particular observations of interest: (1) several of the routines involve the scanning or evaluation of fields (e.g., DLENGTH, EVAL, DCGEN); (2) several other routines involve the processing of tables by storing or searching (e.g., POTGET1, POTGET2, MOTGET1, MOTGET2, LTSTO, STSTO, STGET). The section of this book dealing with the implementation of compilers (Chapter 8) will discuss techniques for parsing fields and evaluating arithmetic expressions, many of which are also applicable to the functional modules of the assembler.

Difference Between

Difference between compiler and interpreter

Sr. No.	Compiler	Interpreter
1	Scans the entire program first and then translate it into machine code	Translate the program line-by-line.
2	Convert the entire program to machine code; when all the syntax errors have been removed execution takes place.	Each time the program is executed, every line is checked for syntax error and then converted to equivalent machine code
3	Execution time is less	Execution time is more
4	Machine code can be saved and used; source code and compiler no longer needed.	Machine code cannot be saved; interpreter is always required for translation.
5	Since source code is not required tampering with the source code is not possible	Source code can be easily modified and hence no security of programs.
6	Slow for debugging	Fast for debugging

Difference between process and Programs

Sr. No.	Process	Programs
1	A process is an instance of program in execution	Program is a set of instructions written to carry out a particular task
2	Process is a dynamic concept	Program is a static concept
3	A process is termed as an 'active entity' since it is always stored in the main memory and disappears if the machine is power cycled. Several processes may be associated with a same program.	A program is an executable file residing on the disk (secondary storage) in a directory
4		It is read into the primary memory and executed by the kernel.
5	A **process is the actual execution** of those instructions.	A computer **program is a passive collection** of instructions;

Difference between multiprogramming and multiprocessing

Sr. No.	Multiprocessing	Multiprogramming
1	Multiprocessing refers to processing of multiple processes at same time by multiple CPUs.	Multiprogramming keeps several programs in main memory at the same time and execute them concurrently utilizing single CPU.
2	It utilizes multiple CPUs.	It utilizes single CPU.
3	It permits parallel processing.	Context switching takes place.
4	Less time taken to process the jobs.	More Time taken to process the jobs.
5	It facilitates much efficient utilization of devices of the computer system.	Less efficient than multiprocessing.
6	Usually more expensive.	Such systems are less expensive.

Difference between Open Subroutine and Closed Subroutine

Sr. No.	Open Subroutine	Closed Subroutine
1	Open subroutine is one whose code inserted into the main program	Close subroutine can be stored outside the main routine and control transfer to the subroutine
2	If some open subroutine where called four times. It would appear in four different placed in the calling program	Close subroutine perform transfer of control and transfer of data
3	Arguments are passed in the registers that are given as arguments to the subroutine.	Arguments may be placed in registers or on the stack
4	Open Subroutines are very efficient with no wasted instructions	A subroutine also allows you to debug code once and then ensure that all future instantiations of the code will be correct
5	Open Subroutines are very flexible and can be as general as the program wishes to make them	Any register that the subroutine uses must first be **saved** and then **restored** after the subroutine completes execution

Difference between Pure Procedure and Impure Procedure

Sr. No.	Pure Procedure	Impure Procedure
1	A pure procedure does not modify itself.	Procedures that modify themselves are called impure procedures.
2	It can be shared by multiple processors	Other program finds them difficult to read and moreover they cannot shard by multiple processors.
3	Pure procedures are readily reusable.	Impute procedures are not readily reusable.
4	To ensure that the instructions are the same each time a program is used.	Each processor executing an impure procedure modifies its contents.
5	Writing such procedure is a good programming practice	Writing such procedure is a poor programming practice